UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
DEPARTAMENTO DE INFORMÁTICA APLICADA
INF01120 - TÉCNICAS DE CONSTRUÇÃO DE PROGRAMAS
PROFESSOR: Marcelo Soares Pimenta (*mpimenta@inf.ufrgs.br*)

Enunciado do Trabalho Prático – FASE 2

Introdução

Este trabalho é feito em grupo, a princípio o mesmo grupo comunicado ao professor. Se houver mudança avise IMEDIATAMENTE o professor (via moodle).

Descrição:

O objetivo do trabalho continua o mesmo: a definição, a implementação, teste e depuração de um GERADOR DE MÚSICA A PARTIR DE TEXTO, tal como na Fase 1.

MAS HÁ MUDANÇAS IMPORTANTES NA DEFINIÇÃO:

MUDANÇA 1): a entrada do texto pode ser via teclado ou via leitura em um arquivo tipo TXT ou ambos (p.ex. ler do arquivo e editar no campo editável) ;-) . Se o arquivo texto for modificado, deve ser possível salvá-lo substituindo o original.
MUDANÇA 2: : Além da saída sonora, deve ser possível também salvar o arquivo MIDI gerado. O grupo decide (i) se o nome seguirá um formato padrão ou será escolhido pelo usuário; e (ii) se o arquivo será gravado num diretório padrão ou a critério do usuário.
MUDANÇA 3: O conjunto de instrumentos do software deve ser escolhido entre os instrumentos do padrão General Midi. Este padrão tem 128 instrumentos (ver https://pt.wikipedia.org/wiki/General_MIDI). O conjunto de vocês deve conter NO MÍNIMO 25 instrumentos, incluindo os fabulosos Pipe Organ ou Church Organ (General Midi 20), e Tubular Bells (General Midi 15). Neste conjunto mínimo devem estar os instrumentos citados neste enunciado (ver abaixo).
MUDANÇA 4 (a maior das mudanças: mudança no mapeamento):

Os caracteres A até G continuam representando respectivamente as notas Lá, Si, Dó, Ré, Mi, Fá, Sol (G). Mas retiramos o H e acrescentamos os acidentes Sustenido (#) e Bemol(b - <B minúsculo>).
Neste novo mapeamento, para conseguir representar uma música de forma mais rica, cada nota terá uma duração associada a ela.
As figuras de duração modernas que constam de uma partitura são (ver Wikipedia):

- Semibreve: É a nota de maior duração na grafia atual. Ela vale o tempo inteiro (1). Identificação para este trabalho: 1
- Mínima: Vale a metade do tempo da semibreve (1/2). São necessárias duas mínimas para somar uma semibreve.). Identificação para este trabalho: 2
- Semínima: Vale a metade da mínima ou um quarto da semibreve (1/4). É o pulso mais comum na maioria das músicas.). Identificação para este trabalho: 3
- Colcheia: Vale a metade da semínima (1/8).). Identificação para este trabalho: 4
- Semicolcheia: Vale a metade da colcheia (1/16).). Identificação para este trabalho: 5
- Fusa: Vale a metade da semicolcheia (1/32).). Identificação para este trabalho: 6
- Semifusa: Vale a metade da fusa (1/64). É a nota mais rápida usada regularmente hoje em dia.). Identificação para este trabalho: 7

Para referenciar uma duração de uma nota, adotamos uma identificação para cada uma destas figuras (ver "Identificação para este trabalho:" para cada figura de duração acima).

Então, se queremos uma nota Lá com duração de semínima, escrevemos "A:3", pois A é o caractere que representa a nota Lá, e a duração de semínima é representada pela identificação de duração 3.

Além desta inclusão de durações a cada nota, iremos propor as seguintes modificações no mapeamento original:

1)Interpretação de linhas como vozes: O arquivo de texto pode conter múltiplas linhas. Cada linha será processada separadamente e atribuída a uma voz diferente no Gerador de Música (V0, V1, V2, ...). A primeira linha é V0, segunda V1, etc. Isso permite polifonia. NESTE TRABALHO, teremos 4 vozes, numeradas de 0 a 3: Voz0, Voz1, Voz2 e Voz3.

Uma Oitava para cada voz: As oitavas agora serão usadas para representar diferentes vozes e assim cada voz terá uma oitava base diferente para representar registros distintos. Por exemplo:

Voz 0: oitava 6 (soprano)

Voz 1: oitava 5 (alto)

Voz 2: oitava 4 (tenor)

Voz 3: oitava 3 (baixo)

Dentro de cada linha (cada voz), as notas podem ainda ser modificadas pelo caractere '?' que sobe UMA oitava, e pelo caractere 'V' que desce UMA oitava mas agora ambos relativos à oitava base da voz.

Entradas sucessivas: Para simular uma fuga, as vozes podem começar em momentos diferentes. Podemos usar um caractere especial no início da linha para indicar o atraso em beats. Por exemplo, se a linha começar com um número entre colchetes seguido de espaço, esse número indica quantos beats de silêncio antes de começar. Ou podemos usar uma sintaxe como "[4]" no início, que significa um delay de 4.

Volume: O volume pode ser usado para dar ênfase ao tema. Por exemplo, quando o sujeito entra, volume máximo; quando é um episódio, volume mais baixo.

O BPM de uma música é GLOBAL, ou seja, vale para toda a música. Então é necessário prever um valor inicial de BPM global.
O andamento pode variar entre seções. Por exemplo, '>' acelera, '<' desacelera e se aplica a todas as vozes.

Instrumentos: Vamos atribuir instrumentos diferentes a cada voz, por exemplo, voz 0: cravo, voz 1: cordas, etc. Podemos usar instrumentos de época, como cravo (GM #6), órgão (GM #20), cordas (GM #48), etc. Cada voz deve ter um instrumento diferente. Então NÃO é necessário prever um instrumento inicial global e sim estabelecer um por default e permitir (por instruções do mapeamento) a troca de instrumento em cada linha.

O texto é lido linha por linha. Cada linha representa uma voz independente.

Para cada linha, o processamento dos caracteres segue as regras originais (notas, silêncios, mudanças de instrumento, volume, oitava), mas com algumas adaptações:

A oitava inicial de cada linha é determinada pela sua posição: linha 1 (primeira) oitava 6, linha 2 oitava 5, linha 3 oitava 4, linha 4 oitava 3, e assim por diante (ciclando se necessário). Então NÃO é necessário prever uma oitava inicial global e sim estabelecer uma por default para cada linha e permitir (por instruções do mapeamento) a troca de oitava em cada linha.

O volume inicial de cada linha pode ser diferente (por exemplo, linha 1 volume 80, linha 2 volume 60, etc.) para dar profundidade.

O instrumento inicial de cada linha pode ser diferente (ex: linha 1: cravo, linha 2: órgão, etc.)

Para simular entradas sucessivas, podemos permitir que cada linha tenha um atraso inicial especificado por um número no começo da linha, entre colchetes. Exemplo: "[4] C:2 D:2 E:3" significa que a linha começa após 4 beats de pausa (delay de 4 beats). Se não houver, começa imediatamente.

RESUMO DAS Principais Mudanças no mapeamento original

Interpretação de linhas como vozes
Cada linha do arquivo de texto representa uma voz independente. As vozes são numeradas sequencialmente (V0, V1, V2, ...) e serão tocadas simultaneamente, criando polifonia.

Oitava base por voz
Para diferenciar os registros (soprano, alto, tenor, baixo), cada voz recebe uma oitava inicial diferente:

Voz 0: oitava 6
Voz 1: oitava 5
Voz 2: oitava 4
Voz 3: oitava 3
A partir da quarta voz, o ciclo se repete (voz 4 = oitava 6, etc.).

Volume base por voz
Para dar profundidade, cada voz começa com um volume distinto (valores de 0 a 127):
Voz 0: 100
Voz 1: 80
Voz 2: 60
Voz 3: 40
(ciclo similar)

Instrumento base por voz (Sugestão de timbres que remetem ao período barroco):
Voz 0: Cravo (GM 6)
Voz 1: Órgão (GM 20)
Voz 2: Piano (GM 0)
Voz 3: Fagote (GM 70)

Volume, Instrumentos e oitavas de cada linha podem ser alterados por comandos durante a execução.

Atraso de entrada
No início de cada linha, pode-se especificar um atraso em beats usando a sintaxe [n] (ex.: [4]). A voz começará após n pausas. Se não houver indicação, o atraso é zero.

Controle de andamento: Esses caracteres podem aparecer em qualquer posição e afetam toda a peça.

> : aumenta o BPM global em 10 (acelera)

< : diminui o BPM global em 10 (desacelera)

Outras letras (vogais O,I,U e consoantes não classificadas – incluindo agora a H, retirada da lista de notas):
Seguem a regra original, ou seja, se o caractere anterior processado era uma nota, repete a última nota tocada; caso contrário, pausa.

Nova linha: Ignorada (já serve como separador de vozes).

O professor irá fornecer – via moodle - alguns exemplos de textos para serem mapeados mas nada impede que os alunos elaborem seus exemplos de textos (como entretenimento ou como forma de testar o seu software).

= =

1a Parte:

ACRESCENTAR ESTES REQUISITOS À lista (informal, em português) dos requisitos ELABORADA NA FASE 1. Sugere-se que cada requisito continue sendo explicitamente rotulado como FUNCIONAL ou NAO-FUNCIONAL.

2a Parte:

Nesta parte, o grupo deve RE-PROJETAR A PARTE FUNCIONAL do sistema, preferencialmente ACRESCENTANDO NOVAS Classes ÀS CLASSES DEFINIDAS NA FASE 1.

3ª Parte:

Nesta parte, o grupo deve RE-PROJETAR A INTERFACE COM O USUÁRIO do sistema (o croqui) INCORPORANDO ELEMENTOS PARA DAR SUPORTE AO QUE FOI SUGERIDO NESTA FASE 3. LEMBREM-SE DE QUE a documentação deverá incluir uma impressão de cada tela/unidade de apresentação por página, seguida das explicações/argumentações que justifiquem seu *layout* e seu comportamento.

4ª Parte:

Nesta parte, o grupo deve IMPLEMENTAR um protótipo do sistema projetado nas partes anteriores. Não foi definida *a priori* nenhuma tecnologia para ser usada mas o grupo pode alternativamente escolher a linguagem e a plataforma mais adequados para seu desenvolvimento, obviamente compatível com os elementos e restrições definidos pelo professor. Reuso de algum código existente é tolerado, mas deve ser sinalizado na documentação e as razões de sua utilização deverão ser explicadas ao professor na documentação entregue.. Implementar o protótipo com o máximo possível de funções que foram projetadas. Não esquecer de TESTAR e DEPURAR o que foi implementado ANTES de demonstrá-lo ao professor.

5ª Parte:

Nesta Parte, o grupo deve APRESENTAR as características principais de seu projeto e DEMONSTRAR o sistema para o professor. No final do semestre, cada grupo irá apresentar as idéias principais de seu trabalho para a turma (20 min no máximo por apresentação). Além disto, a versão implementada do sistema deverá ser apresentada - em data a ser definida e divulgada - ao professor em uma sessão exclusiva para cada grupo, onde o professor irá inspecionar o código do software. Assume-se que a documentação final do trabalho será entregue ao professor no máximo até o dia da demonstração. O professor não apenas fará uso da documentação para utilizar o sistema e avaliá-lo mas também poderá inspecionar o GitHub do grupo e questionar oralmente os componentes do grupo a respeito de todas as partes do trabalho. Deve ficar clara nas respostas a participação e o engajamento de todos os indivíduos no trabalho do grupo. Como combinado em aula, os grupos podem usar IA mas DEVEM ser capazes de entender e explicar o código apresentado.

Observações:

O trabalho a ser entregue inclui a documentação associada a TODAS as partes desta fase e deve conter também a identificação do grupo (número do grupo, nomes de todos componentes) e todas as suposições feitas durante a realização do trabalho. Definições mais precisas que porventura sejam necessárias serão acrescentadas no decorrer do semestre. Em caso de dúvidas, consulte primeiro a bibliografia disponível (preferencialmente), depois pesquise em outras bibliografias (use a biblioteca e a Internet) e, em caso de necessidade, consulte o professor nas aulas de mentoria ou via e-mail. A disciplina prevê algumas aulas de mentoria para acompanhamento dos projetos e esclarecimento das dúvidas surgidas. As datas são divulgadas em aula e via moodle. Em caso de reunião fora do período de aula, lembre-se de agendar com o professor (via email) para garantir que será atendido. Sem agendamento prévio, a prioridade do professor será sempre atender aos compromissos agendados previamente. Bom trabalho!!

Datas importantes SERÃO DEFINIDAS EM AULA

- Entrega da Documentação: Documentos e código fonte desta fase. AULAS DE MENTORIA: serão agendadas em sala de aula algumas aulas para resolução de dúvidas ou discussão do trabalho com os colegas e o professor; uma vez decididas, as datas serão divulgadas via moodle da disciplina.
- Demonstração– a partir da Entrega. Na prática, a data de demonstração deve ser agendada com o professor.